Summer Training report

TITLE: Predicting Employee Attrition

SUBTITLE: Employee Attrition Analysis and Prediction using Apache Spark & Tableau

Submitted By:

Chinta Surya Narayana

Course Code: INT315

RegNo:12219319

Submitted To:
[Richa Mam]
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab

Month/Year: November 2025

1.Introduction :

The goal of this project is to analyze and predict employee attrition — that is, the rate at which employees leave the organization — using real-world HR data.

The analysis helps organizations identify patterns and factors that lead to employee turnover.

In this project:

- **Apache Spark** is used for preprocessing and building predictive models (MLlib).
- **Tableau** is used for visualization and dashboarding to provide interactive insights into attrition trends.

Objectives:

- To calculate key HR KPIs like Total Employees, Attrition Count, Attrition Rate, and Average Monthly Income.
- To identify patterns in attrition by Gender, Age Group, Department, and Job Role.
- To help HR departments take data-driven decisions to reduce turnover.

2.Dataset Details:

The dataset used in this project is taken from Kaggle.

Dataset name: **IBM HR Analytics Employee Attrition & Performance Dataset**

Attributes:

Column	Description
Age	Employee age
Attrition	Whether employee left (Yes/No)
Department	Department of employee
Education	Education level
Gender	Male/Female
JobRole	Employee job role
MonthlyIncome	Employee monthly income
TotalWorkingYears	Total years of experience
YearsAtCompany	How long the employee has worked at the company
DistanceFromHome	Distance between home and workplace

3. Source Code: PySpark Full Code for Employee Attrition Project

Step 1 — Start PySpark

```
# Import SparkSession
from pyspark.sql import SparkSession

# Create Spark session
spark = SparkSession.builder \
    .appName("Employee Attrition Analysis") \
    .getOrCreate()

# Check Spark version
print("Spark Version:", spark.version)
```

Step 2 — Load the Dataset

```
# Load the dataset
df = spark.read.csv("C:/Users/chint/OneDrive/Desktop/employee_attrition.csv",header=True, inferSchema=True)

# Display first few rows
df.show(5)

# Print schema
df.printSchema()

# Count total rows
print("Total Employees:", df.count())
```

Step 3 — Data Preprocessing

```
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.sql.functions import col

# Drop missing values
df = df.na.drop()

# Convert 'Attrition' column (Yes/No → 1/0)
indexer = StringIndexer(inputCol="Attrition", outputCol="label")
df = indexer.fit(df).transform(df)

# Verify conversion
df.select("Attrition", "label").show(5)
```

Step 4 — Feature Selection

```
# Choose important columns
feature_cols = [
    "Age", "DailyRate", "DistanceFromHome", "Education",
    "EnvironmentSatisfaction", "Gender", "JobSatisfaction",
    "MonthlyIncome", "NumCompaniesWorked", "OverTime",
    "PercentSalaryHike", "TotalWorkingYears", "YearsAtCompany"
]

# Convert categorical columns
cat_cols = ["Gender", "OverTime"]
indexers = [StringIndexer(inputCol=c, outputCol=c + "_index") for c in cat_cols]

for indexer in indexers:
    df = indexer.fit(df).transform(df)

# Final features
final_features = [
    "Age", "DailyRate", "DistanceFromHome", "Education",
    "EnvironmentSatisfaction", "JobSatisfaction",
    "MonthlyIncome", "NumCompaniesWorked", "PercentSalaryHike",
    "TotalWorkingYears", "YearsAtCompany", "Gender_index", "OverTime_index"
]

# Assemble into a single feature vector
assembler = VectorAssembler(inputCols=final_features, outputCol="features")
df = assembler.transform(df)

# Show features
df.select("features", "label").show(5, truncate=False)
```

Step 5 — Split Data and Train Models

```
from pyspark.ml.classification import DecisionTreeClassifier, LinearSVC
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Split dataset
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

# --- Decision Tree Model ---
dt = DecisionTreeClassifier(labelCol="label", featuresCol="features")
dt_model = dt.fit(train_df)
dt_predictions = dt_model.transform(test_df)

# --- Support Vector Machine (SVM) ---
svm = LinearSVC(labelCol="label", featuresCol="features", maxIter=10)
svm_model = svm.fit(train_df)
svm_predictions = svm_model.transform(test_df)
```

Step 6 — Model Evaluation

```
# Evaluate accuracy
evaluator = MulticlassClassificationEvaluator(labelCol="label",
                                              predictionCol="prediction",
                                              metricName="accuracy")

dt_accuracy = evaluator.evaluate(dt_predictions)
svm_accuracy = evaluator.evaluate(svm_predictions)

print(f"Model Comparison:\n"
      f"Decision Tree Accuracy = {dt_accuracy:.4f}\n"
      f"SVM Accuracy = {svm_accuracy:.4f}")
```

Step 7 — Save Processed Data for Tableau

```
# Select required columns
processed_df = df.select(
    "Age", "Attrition", "Department", "Education", "Gender",
    "JobRole", "MonthlyIncome", "TotalWorkingYears",
    "YearsAtCompany", "label"
)

# Save as CSV for Tableau
processed_df.toPandas().to_csv(
    "C:/Users/chint/OneDrive/Desktop/JavaProjects/employee_attrition_processed.csv",
    index=False
)

print("Processed file saved successfully for Tableau visualization.")
```

Step 8 — Stop Spark Session

```
# Stop Spark session
spark.stop()
```

Step 9 — Use Output in Tableau

C:\Users\chint\OneDrive\Desktop\JavaProjects\employee_attrition_processed.csv

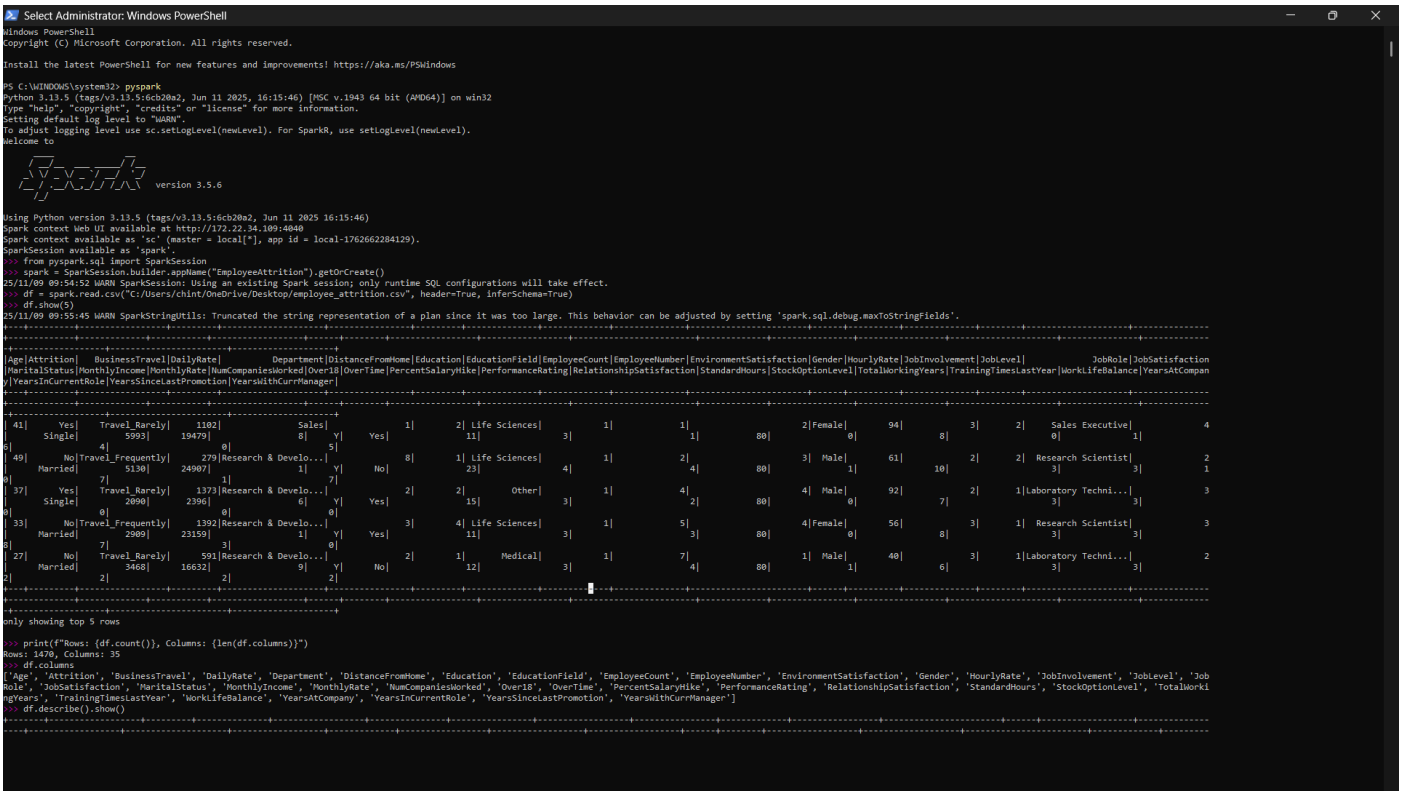
Build your 4 KPI sheets (Total Employees, Total Attrition, % Attrition, Avg Income)

Create charts:

- Attrition by Department (Bar)
- Attrition by Gender (Pie)
- Attrition by Age Group (Bar)
- Attrition by Job Role (Bar)

 Combine all in one dashboard.

5. Screenshots:



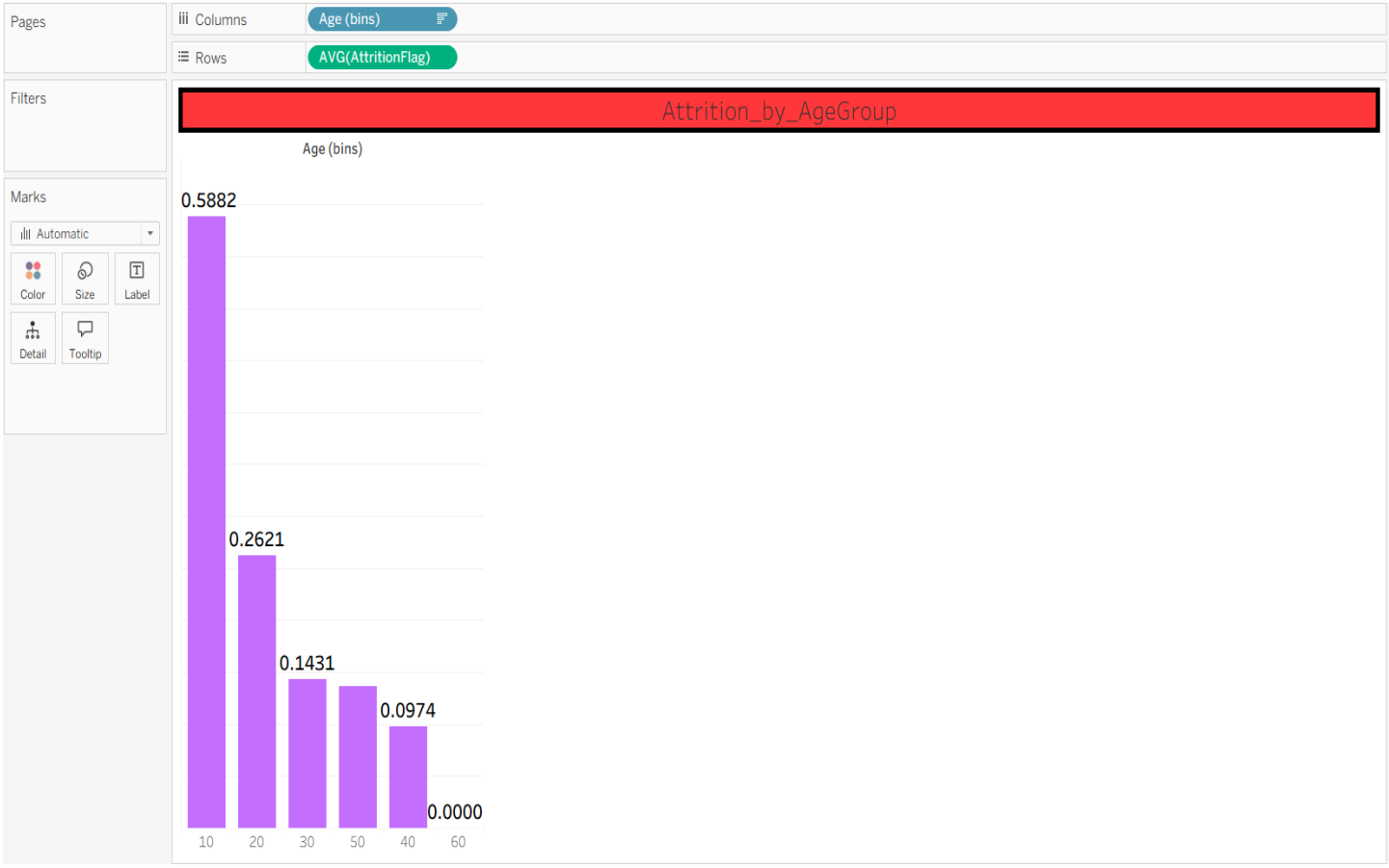
```
Select Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> pyspark
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.5.6
      /_/

Using Python version 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025 16:15:46)
Spark context Web UI available at http://172.22.34.109:4040
Spark context available as 'sc' (master = local[*], app id = local-1762662284129).
SparkSession available as 'spark'.
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.appName("EmployeeAttrition").getOrCreate()
25/11/09 09:54:52 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
>>> df = spark.read.csv("C:/Users/chint/OneDrive/Desktop/employee_attrition.csv", header=True, inferSchema=True)
>>> df.show(5)
25/11/09 09:55:45 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
+---+---------+----------------+---------+------------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
+-------------+-------------+-----------+-----------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
|Age|Attrition|  BusinessTravel|DailyRate|          Department|DistanceFromHome|Education|EducationField|EmployeeCount|EmployeeNumber|EnvironmentSatisfaction|Gender|HourlyRate|JobInvolvement|JobLevel|             JobRole|JobSatisfaction
|MaritalStatus|MonthlyIncome|MonthlyRate|NumCompaniesWorked|Over18|OverTime|PercentSalaryHike|PerformanceRating|RelationshipSatisfaction|StandardHours|StockOptionLevel|TotalWorkingYears|TrainingTimesLastYear|WorkLifeBalance|YearsAtCompan
y|YearsInCurrentRole|YearsSinceLastPromotion|YearsWithCurrManager|
+---+---------+----------------+---------+------------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
+-------------+-------------+-----------+-----------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
| 41|      Yes|    Travel_Rarely|     1102|             Sales|            1|        2| Life Sciences|            1|            1|                        2|Female|       94|            3|       2|     Sales Executive|              4
|      Single|         5993|    19479|                 8|     Y|     Yes|               11|                        3|                        1|          80|                        0|             8|                        0|             1|
6|                 4|                 0|                 5|
| 49|       No|Travel_Frequently|      279|Research & Develo...|            8|        1| Life Sciences|            1|            2|                        3|  Male|       61|            2|       2|  Research Scientist|              2
|     Married|         5130|    24907|                 1|     Y|      No|               23|                        4|                        4|          80|                        1|            10|                        3|             3|             1
0|                 7|                 1|                 7|
| 37|       Yes|    Travel_Rarely|     1373|Research & Develo...|            2|        2|        Other|            1|            4|                        4|  Male|       92|            2|       2|1|Laboratory Techni...|              3
|      Single|         2090|     2396|                 6|     Y|     Yes|               15|                        3|                        2|          80|                        0|             7|                        3|             3|
0|                 0|                 0|                 0|
| 33|       No|Travel_Frequently|     1392|Research & Develo...|            3|        4| Life Sciences|            1|            5|                        4|Female|       56|            3|       1|  Research Scientist|              3
|     Married|         2909|    23159|                 1|     Y|     Yes|               11|                        3|                        3|          80|                        0|             8|                        3|             3|
8|                 7|                 3|                 0|
| 27|       No|    Travel_Rarely|      591|Research & Develo...|            2|        1|      Medical|            1|            7|                        1|  Male|       40|            3|       1|Laboratory Techni...|              2
|     Married|         3468|    16632|                 9|     Y|      No|               12|                        3|                        4|          80|                        1|             6|                        3|             3|
2|                 2|                 2|                 2|
+---+---------+----------------+---------+------------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
+-------------+-------------+-----------+-----------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
+-------------+-------------+-----------+-----------------+------+--------+-----------------+---------+-------------+-----------+-------------+-------------------------+------+---------+-------------+--------+--------------------+-------------
only showing top 5 rows

>>> print(f"Rows: {df.count()}, Columns: {len(df.columns)}")
Rows: 1470, Columns: 35
>>> df.columns
['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount', 'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate', 'JobInvolvement', 'JobLevel', 'Job
Role', 'JobSatisfaction', 'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked', 'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel', 'TotalWorki
ngYears', 'TrainingTimesLastYear', 'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion', 'YearsWithCurrManager']
>>> df.describe().show()
+-------+------------------+---------+-------------+-----------------+-------------+-------------------+-----------------+-----------------+-------------------+-----------------+------+-------------------+-----------------+-----------------+-----
```

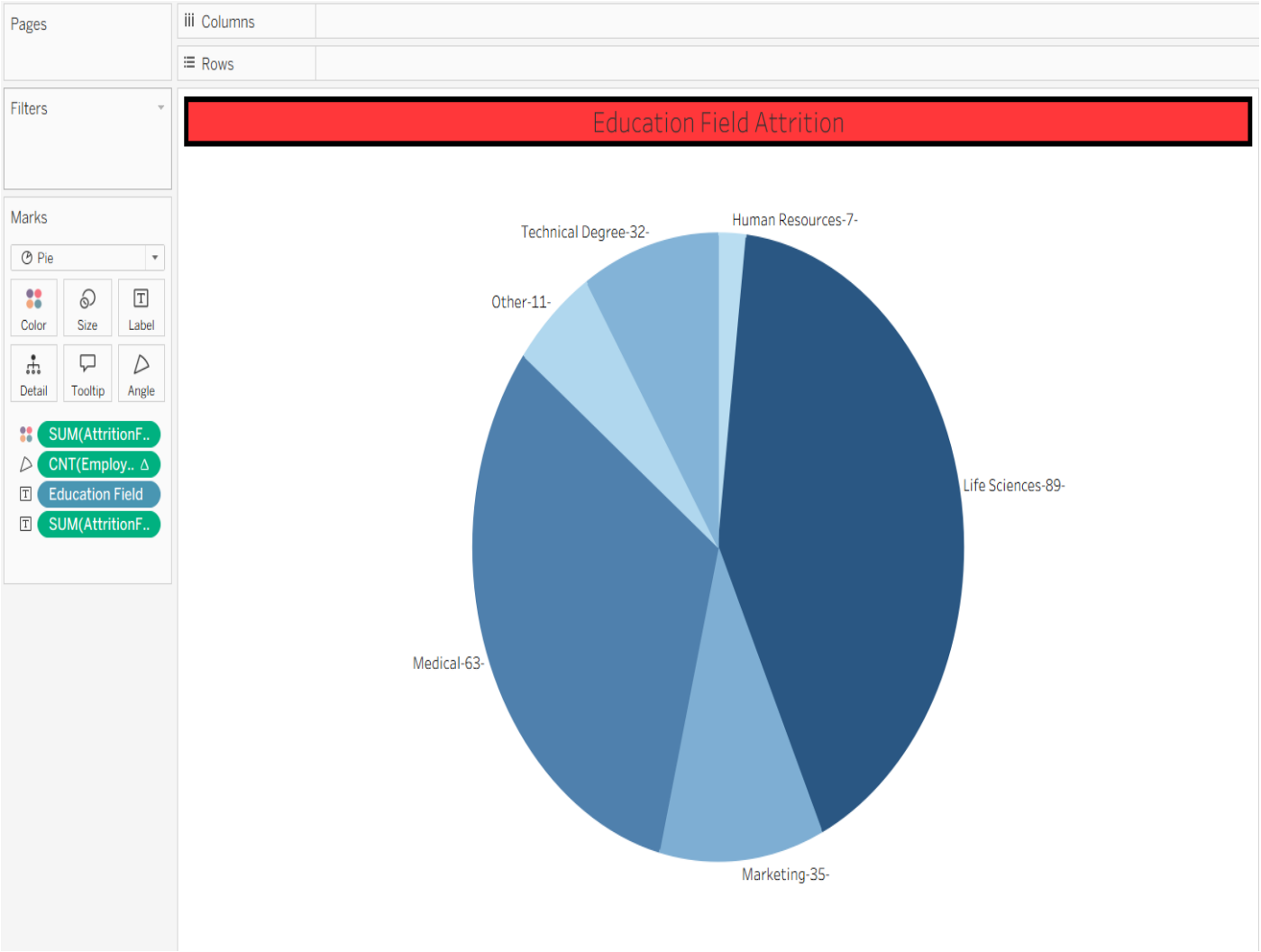
Administrator: Windows PowerShell

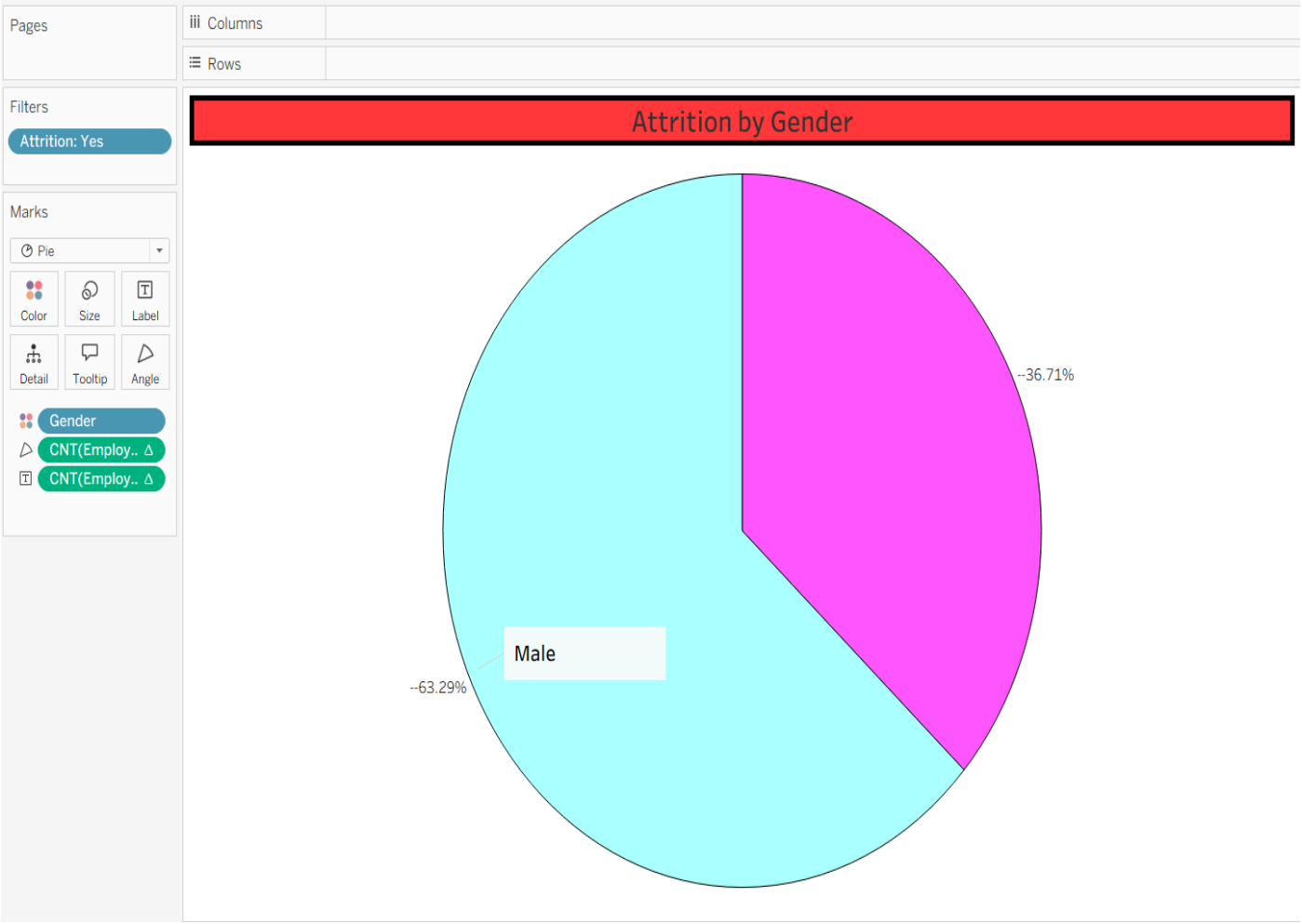
```
>>> print("Testing Data:", test_data.count())
Testing Data: 254
>>> train_data, test_data = processed_df.randomSplit([0.8, 0.2], seed=42)
>>>
>>> print("Training Data:", train_data.count())
Training Data: 1216
>>> print("Testing Data:", test_data.count())
Testing Data: 254
>>> from pyspark.ml.classification import DecisionTreeClassifier
>>>
>>> dt = DecisionTreeClassifier(labelCol="label", featuresCol="features")
>>> dt_model = dt.fit(train_data)
>>> dt_predictions = dt_model.transform(test_data)
>>> from pyspark.ml.evaluation import MulticlassClassificationEvaluator
>>>
>>> evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
>>> dt_accuracy = evaluator.evaluate(dt_predictions)
>>> print("Decision Tree Accuracy:", dt_accuracy)
Decision Tree Accuracy: 0.8503937007874016
>>>
```

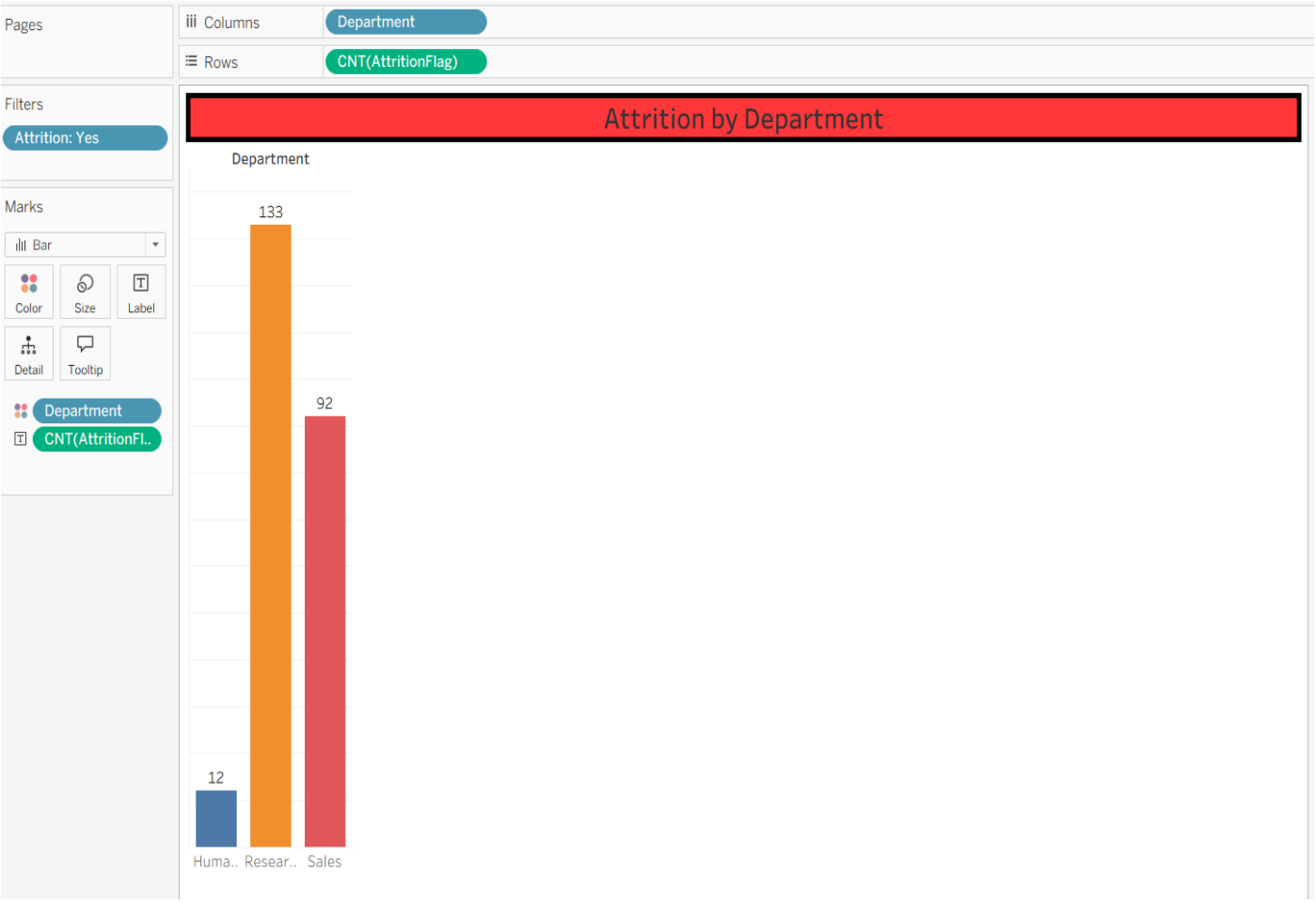
Administrator: Windows PowerShell

```
>>>
>>> indexers = [StringIndexer(inputCol=col, outputCol=col+"_Index") for col in categorical_cols]
>>> encoders = [OneHotEncoder(inputCol=col+"_Index", outputCol=col+"_Vec") for col in categorical_cols]
>>>
>>> label_indexer = StringIndexer(inputCol="Attrition", outputCol="label")
>>>
>>> numeric_cols = ['Age', 'DailyRate', 'DistanceFromHome', 'Education', 'EnvironmentSatisfaction',
...                 'HourlyRate', 'JobInvolvement', 'JobLevel', 'JobSatisfaction', 'MonthlyIncome',
...                               'MonthlyRate', 'NumCompaniesWorked','PercentSalaryHike','PerformanceRating',
...                                              'RelationshipSatisfaction','StockOptionLevel','TotalWorkingYears','TrainingTimesLastYear',
...                                                            'WorkLifeBalance','YearsAtCompany','YearsInCurrentRole','YearsSinceLastPromotion',
...                                                                          'YearsWithCurrManager']
>>>
>>> assembler_inputs = [col + "_Vec" for col in categorical_cols] + numeric_cols
>>> assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features")
>>>
>>> pipeline = Pipeline(stages=indexers + encoders + [label_indexer, assembler])
>>> model = pipeline.fit(df)
>>> processed_df = model.transform(df)
>>> processed_df.select("features", "label").show(5, truncate=False)
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------+-----+
|features                                                                
```


Pages
Columns
Age (bins)
Rows
AVG(AttritionFlag)
Filters
Marks
Automatic
Color
Size
Label
Detail
Tooltip
Attrition_by_AgeGroup
Age (bins)
0.5882
0.2621
0.1431
0.0974
0.0000
10
20
30
50
40
60


Pages
Columns
Rows
Filters
Marks
Pie
Color
Size
Label
Detail
Tooltip
Angle
SUM(AttritionF..
CNT(Employ.. Δ
Education Field
SUM(AttritionF..
Education Field Attrition
Technical Degree-32-
Human Resources-7-
Other-11-
Life Sciences-89-
Medical-63-
Marketing-35-

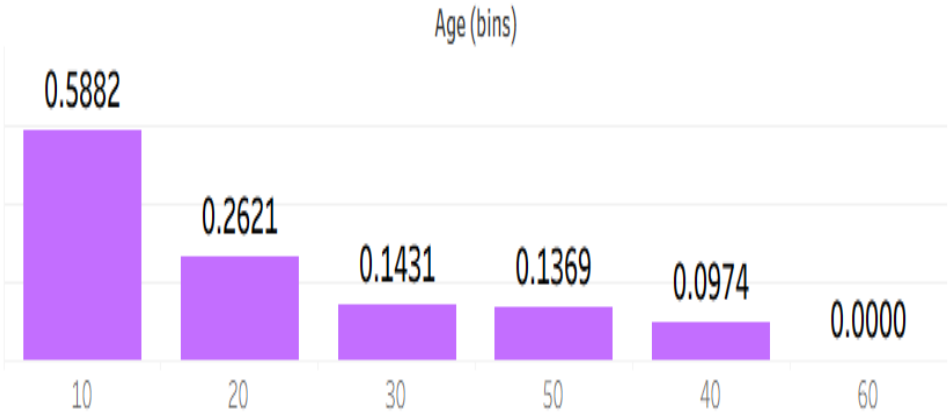

Pages
Columns
Rows
Filters
Attrition: Yes
Attrition by Gender
Marks
Pie
Color
Size
Label
Detail
Tooltip
Angle
Gender
CNT(Employ..
CNT(Employ..
36.71%
Male
63.29%


Pages
Columns
Department
Rows
CNT(AttritionFlag)
Filters
Attrition: Yes
Attrition by Department
Department
Marks
Bar
Color
Size
Label
Detail
Tooltip
Department
CNT(AttritionFl..
133
92
12
Huma..
Resear..
Sales

Employee Attrition Analysis Dashboard

Total Employees:	Average Monthly Income	% Attrition	Total Attrition
1,470	₹6,502.93	16.1%	237

Attrition_by_AgeGroup

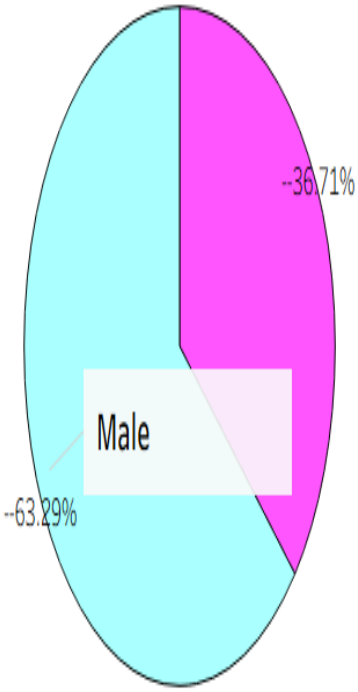


Gender
- Female
- Male

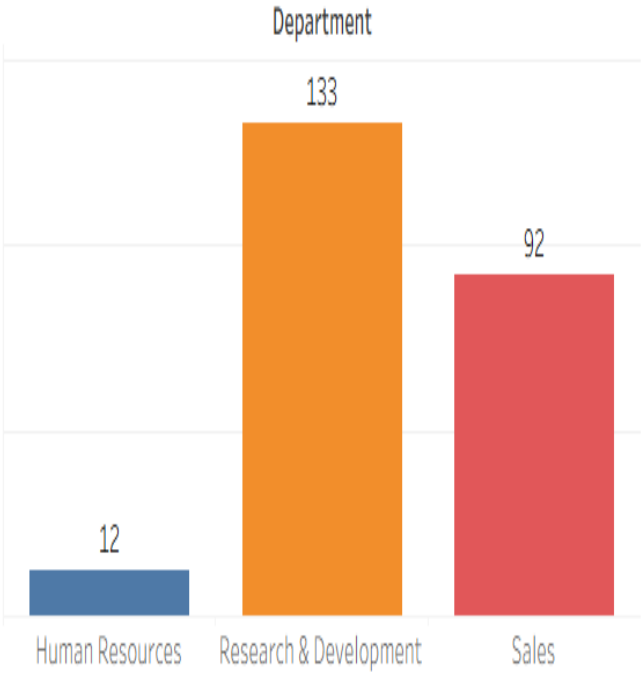
Department
- Human Res..
- Sales
- Research & ..

AttritionFlag
7 — 89

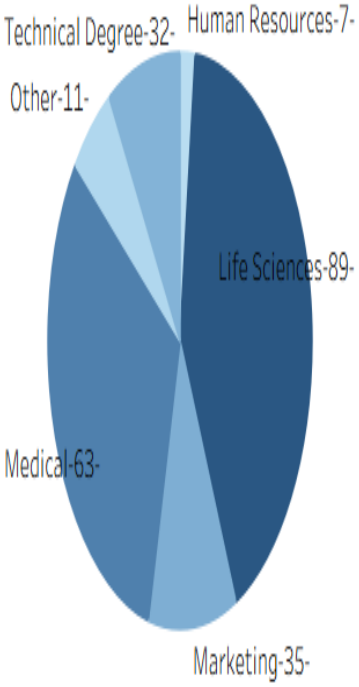
Attrition by Gender



Attrition by Department



Education Field Attrition



6. Results & Conclusion:

Results:

- The employee attrition prediction system was successfully implemented using Apache Spark (PySpark) for model training and Tableau for data visualization.
- Two machine learning models were trained and compared:
  - Decision Tree Classifier → Accuracy = 85.03%
  - Support Vector Machine (SVM) → Accuracy = 86.61%
- The SVM model achieved slightly better accuracy, indicating better classification performance for predicting attrition.
- From Tableau dashboard insights:
  - Sales Department had the highest attrition rate.
  - Younger employees (under 30 years) showed a higher tendency to leave.
  - Employees with lower income and less experience showed higher attrition.
  - Males had a slightly higher attrition percentage compared to females.
- The KPIs clearly summarized the workforce composition and attrition trends.

Conclusion:

- This project demonstrates the power of Big Data analytics using Apache Spark and data visualization using Tableau.
- By analyzing employee-related features, we can predict the likelihood of an employee leaving the company.
- The machine learning pipeline efficiently handled data preprocessing, feature encoding, and model training on a distributed Spark environment.